Desafio Prático: Gestão de Estoque da Loja de Roupas

Construção Progressiva: Listas Paralelas, Inserção, Remoção e Fluxogramas

4 Etapas

Aluno(a): ..... **Turma:** ..... **Data:** .....

Parte 1: Criando as Listas Iniciais (Produtos e Preços)

PASSO 1

Crie duas listas sincronizadas: `produtos` (com pelo menos 3 roupas) e `precos` (com os preços correspondentes no mesmo índice).

```
produtos = ["Camiseta Básica", "Calça Jeans", "Jaqueta Corta-Vento"] precos = [49.90, 129.90, 219.90]
```

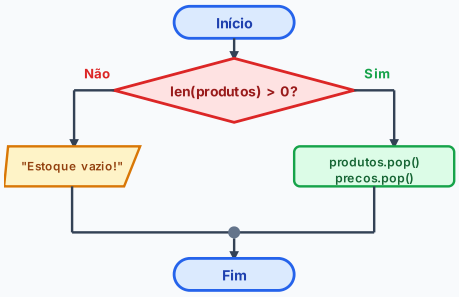
Parte 2: Cadastrando Novos Itens (Leitura e append)

PASSO 2

Peça ao usuário o nome de uma nova peça de roupa e seu preço (com `float`). Em seguida, insira os valores nas listas usando `.append()` e mostre o estoque atualizado.

Parte 3: Fluxograma — Remover o Último Item (pop)

PASSO 3

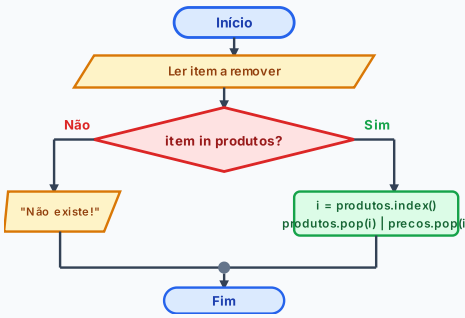


Sua Missão:

- Leia a condição no losango: por que testamos se `len > 0`?
- Implemente o `if/else` com o método `.pop()`.
- Mostre na tela qual foi o produto e o preço removidos.

Parte 4: Fluxograma — Busca e Remoção por Nome

PASSO 4 (FINAL)



Sua Missão:

- Pergunte ao usuário o nome exato da peça a remover.
- Use `if item in produtos:` para checar se ela existe.
- Descubra a posição com `.index()` e remova de ambas as listas com `.pop(indice)`!

Gabarito do Professor: Gestão de Estoque

Código Completo Comentado & Notas de Mediação Pedagógica

Gabarito Oficial

Código Python Completo (Etapas 1, 2, 3 e 4)

Tempo sugerido em aula: 30 a 40 minutos

```
# ================================================================= # ETAPA 1: CRIANDO AS LISTAS INICIAIS # ================================================================= produtos = ["Camiseta Básica", "Calça Jeans", "Jaqueta Corta-Vento"] precos = [49.90, 129.90, 219.90] print("Estoque Inicial:", produtos) print("Preços: ", precos) # ================================================================= # ETAPA 2: ADICIONANDO NOVOS PRODUTOS # ================================================================= novo_produto = input("\n[Adicionar] Nome da peça de roupa: ") novo_preco = float(input("[Adicionar] Preço da peça (R$): ")) produtos.append(novo_produto) precos.append(novo_preco) print(f"Sucesso: '{novo_produto}' adicionado por R$ {novo_preco:.2f}.") # ================================================================= # ETAPA 3: REMOVENDO O ÚLTIMO ITEM (.pop()) COM PROTEÇÃO # ================================================================= if len(produtos) > 0: removido_prod = produtos.pop() removido_val = precos.pop() print(f"\n[Remover Último] Removido: '{removido_prod}' (R$ {removido_val:.2f})") else: print("\n[Aviso] O estoque já está completamente vazio!") # ================================================================= # ETAPA 4: BUSCAR E REMOVER PRODUTO ESPECÍFICO # ================================================================= item_busca = input("\n[Remover Específico] Qual produto deseja remover? ") if item_busca in produtos: # 1. Localiza a posição do produto indice = produtos.index(item_busca) # 2. Remove de ambas as listas pelo índice sincronizado prod_removido = produtos.pop(indice) val_removido = precos.pop(indice) print(f"Sucesso: '{prod_removido}' (R$ {val_removido:.2f}) foi removido.") else: print(f"Erro: O produto '{item_busca}' não foi encontrado no estoque.") print("\n═══ ESTOQUE FINAL ═══") print("Produtos:", produtos) print("Preços: ", precos)
```

💡 Por que trabalhar com duas listas paralelas?

Antes de ensinar Pandas/DataFrames, listas paralelas ensinam o conceito de **índice sincronizado**: o item na posição `0` da lista de produtos tem seu valor na posição `0` da lista de preços.

⚠️ Erro mais comum dos alunos:

Esquecer de remover das **duas** listas! Se o aluno fizer `produtos.remove()` mas esquecer do preço, o estoque perde a sincronia e o casaco passa a ter preço de meia.